SOL PROTOCOL

Level Design: General Info

Sectors, dungeon metrics, content, loot setup and the procedural sector generator

Contents

1 World structure 1
2 Metrics 2
3 Content catalogue 3
4 Building a dungeon 5
5 Loot system 6
6 Procedural sector generator 9

1 World structure

1.1 Sectors

Sectors are small regions of space where the player can manually pilot their ship. Within these areas, players may encounter dungeons, other ships, loot, and additional points of interest.

Each sector is procedurally generated every time the team jumps from the previous one. In some sectors, players can find probes, which are required to initiate the jump to the next galaxy and progress through the game.

Sector metrics

Parameter	Value
Sector radius (max)	400,000 cm
Sector radius (min)	10,000 cm
Minimum distance between objects	10,000 cm
Object scale	1.0 – 2.0
Number of objects to spawn	*To be defined*

1.2 Dungeons

Dungeons are small, self-contained levels scattered throughout the universe. Players can enter them to solve puzzles and collect loot to bring back to their ship.

They are abandoned bases built inside asteroids or small planetoids, left unattended for a long time. They still contain defenses and hidden mechanisms that protect valuable loot, which players must overcome or solve to obtain it.

Design rules

- **Single entrance:** a dungeon is accessible only through an airlock door.

- **Traps:** laser barriers and automated turrets (turrets not yet implemented).
- **Gravity:** players can turn gravity on or off with a specific button, reaching previously inaccessible areas.
- **No combat:** there are no combat encounters of any kind. Players rely solely on solving puzzles to open the way to the loot room.

2 Metrics

All values in centimeters. L = width, H = height, D = depth.

2.1 Player and ship

Parameter	Value
Jump height	H 100 cm
Ship speed	5,000 cm/s
Ship impulse (sprint)	Ship speed × 100

2.2 Walkable space

Element	L	H
Door (minimum)	175	245
Corridor (minimum walkable space)	175	245
Corridor (medium walkable space)	500	500

2.3 Dungeon walls and modules

Module	L	H	D
Basic wall	500	500	25
Small wall	100	500	25
Broken wall	100	180	25
Normal door	250	300	10
Normal door + framework	500	500	25
Small door	175	250	25
Small door + framework	300	500	25
Window	300	200	10
Window + framework	500	500	25

3 Content catalogue

3.1 Spawnable objects

Objects that can be placed in a sector, grouped by category.

Blueprint	Category	Status	In generator
BP_Enemy_HeavyGatling	Enemy	In progress	No
BP_Formation	Enemy	Finished	No
BP_Enemy_Ship_Mid01	Enemy	Finished	No
BP_Enemy_Ship_Mid02	Enemy	Finished	No
BP_ShipStation	Enemy	—	No
BP_PlayerShipInsideSphere_SpawnEnemies	Enemy	Finished	No
BP_AsteroidFieldSphere	Explorable environment	—	No
BP_Asteroid_Build_01	Explorable environment	Finished	No
BP_Asteroid_Build_02	Explorable environment	Finished	No
BP_Planet	Unexplorable environment	—	No
BP_DungeonBase_01	Dungeon	Finished	No
BP_EX01_LaserMazeDungeon	Dungeon	Finished	No
BP_EX_LaserMazeDungeon2	Dungeon	Finished	No
BP_EX_LaserMazeDungeon3	Dungeon	Finished	No
BP_EX_GravityDungeon2	Dungeon	Finished	No
BP_EX_GravityDungeon3	Dungeon	Finished	No
BP_EX03_ParkourDungeon1	Dungeon	Finished	No
BP_EX_ParkourDungeon02	Dungeon	Finished	No
BP_EX_ParkourDungeon03	Dungeon	Finished	No
BP_EX_ParkourDungeon04	Dungeon	Finished	No
BP_EX_ParkourDungeon_Var03	Dungeon	Finished	No
BP_EX_PlatformDungeon01	Dungeon	Finished	No
BP_EX_PlatformDungeon02	Dungeon	Finished	No
BP_EX_PuzzleDungeon01	Dungeon	Finished	No
BP_EX_PuzzleDungeon02	Dungeon	Finished	No
BP_EX_trapDungeon01	Dungeon	Finished	No
BP_RandomMineSpawner	Event	—	No
BP_LootSpawner	Loot	—	No

Other entries in the list, with no category or status yet: Intel gathering, Ship graveyard, Planet explorable, Planet with loot, Ship relict, Raids, Procedural dungeon (temporary name).

3.2 Interactable objects

Grabbable items and loot

Item	Rarity	Purpose
Mod	Rare	Upgrades an existing system on the ship
System chip	Rare	Brought back to the ship to change or acquire new systems
Hull repair sphere	Common – Rare	Repairs part of the damage sustained by the ship
System repair sphere	Common	Repairs all the damage sustained by a ship system or weapon
Missile ammo	Common – Rare	Ammo for the missile launcher
Laser ammo	Common	Ammo for the laser cannon

Linked doors

Element	Behavior
Simple button	When pressed, opens the door or doors linked to it

3.3 Naming conventions

Asset	Pattern	Example
Dungeon exterior	`BP_EX_<Type>Dungeon<Number>`	`BP_EX_TrapDungeon01`
Dungeon interior	`BP_IN_<Type>Dungeon<Number>`	`BP_IN_TrapDungeon01`
Loot data asset	`LOOT_<Name>`	`LOOT_Test`

4 Building a dungeon

4.1 Create a new dungeon

1. Create a new folder in `/All/Game/Space/Dungeons/Dungeons`.
2. Inside the folder, create two blueprints:
   - `BP_DungeonBase`: the exterior, what the player sees outside the dungeon.
   - `BP_DungeonSublevel`: the interior, what the player sees after entering the airlock.
3. Build the dungeon **inside the sublevel**.

> **The interior must fit inside the exterior**
> The dungeon interior MUST stay inside the objects of the DungeonBase. For example, if the dungeon is inside an asteroid, the asteroid must be big enough to contain it.

4.2 Link the interior to the exterior

1. Open the blueprint of the dungeon exterior.
2. In **Components**, select **Interior**.
3. In **Details › Child Actor Component**, set **Child Actor Class** to the dungeon interior.

4.3 Gravity button

To enable gravity, place the gravity blueprint so that its collision box contains every area where gravity is active. To turn it on and off with a button:

1. Create a child actor (any name).
2. Set its class to `BP_TouchButton_NotReplicated`.
3. In **Config › Signal**, write `Dungeon:Toggle:Gravity`.

For more advanced setups, ask Andrea.

4.4 Connect buttons to doors

1. Create two child actors.
2. Door: set the class to `BP_Door_Base` (or any other door).
   - In **Default**, make sure **Autolink** is checked.
   - In **Config**, write a **DoorStateName**. It must be unique within the dungeon.
3. Button: set the class to `BP_TouchButton_NotReplicated`.
   - In **Config › Signal**, write `Dungeon:Toggle:[DoorStateName]`.

4.5 Collision channels

Object	Setup
Static objects that must collide with the ship's hull	CollisionPreset `Hull`

Objects pushed by the ship on impact (physics)	Preset `PhysicsActor`
	`SimulatePhysics = true`
	`EnableGravity = false`

> **Performance**
> Keep the number of physics objects as low as possible.

5 Loot system

5.1 Loot data assets

Loot probability presets are data assets stored in `Space/Data/Loot` and named `LOOT_<Name>`. To create a new one, duplicate an existing preset and empty its *ProbabilitiesPerType* map.

In *ProbabilitiesPerType*, each row sets the spawn probability of one loot type in the preset:

- probabilities must add up to 1;
- if there must be a chance of spawning nothing, use `Loot_Null`.

5.2 Enemy drops

Enemy drops depend on the enemy class. When destroyed, an enemy deploys *NumberOfLootToSpawn* loot actors. To set them up, create a `LOOT_*` data asset and ping a developer with the enemy class, the data asset and the number of loot actors.

> **Note for developers**
> Enemies should have a `BP_EnemyLootDropComponent` with a *LootDataAsset* parameter, where one of the presets above can be set.

5.3 Chest drops

Ready-made chests

Chest	Blueprint
Common loot chest	`BP_LootChest_Common`
Rare loot chest	`BP_LootChest_Rare`

Custom drop with a placeholder

In both cases the child actor class is `BP_DungeonItemPlaceholder`.

Option	Setup
A) Random drop from a loot data asset	Set **Class** to `BP_LootSpawner`. In the **InstancedStruct** field that appears, select `FLootSpawner`. Add the `LOOT_*` data assets you want.
B) One specific actor, bypassing the random loot system	Set **Class** directly to the class of the object to spawn, instead of `BP_LootSpawner`.

Inside dungeons, each `BP_DungeonItemPlaceholder` supports only one spawned loot actor.

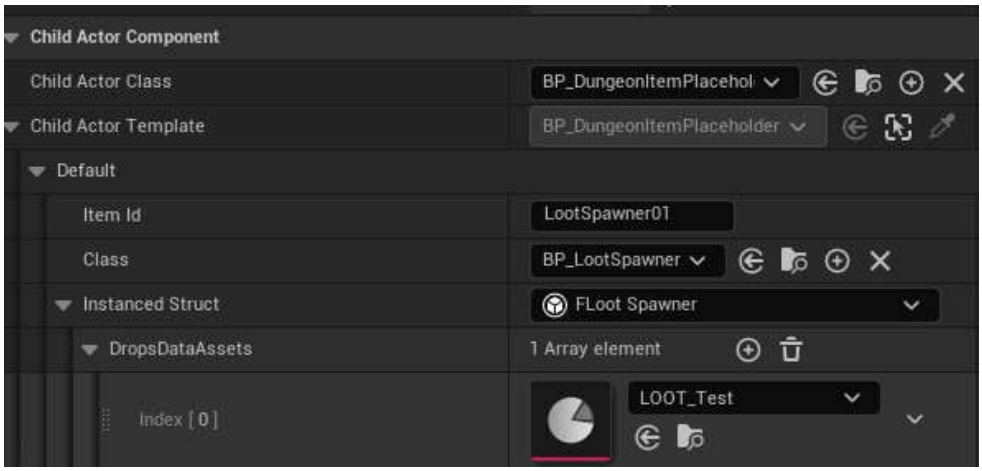


Figure 1 — Option A: placeholder set up with a loot spawner and a data asset

5.4 Random drops in the level

Add a `BP_LootSpawner` to the level and fill its *LootDataAssets* array. At the start of the game it drops one loot item per data asset in the array.

5.5 From loot type to spawned actor

The default values of the struct `S_SpawnableLootActors` define which actor spawns for each loot type. For example, if a data asset specifies the loot type `Loot_Ammo`, the game spawns a `BP_AmmoBox`. There is normally no need to change *HullRepairSphereTypes*, *SystemRepairSphereTypes* or *AmmoBoxes*.

> **Add every new mod to the list**
> When spawning a mod, the loot spawner picks a random one from `S_SpawnableLootActors.ModTypes`. Newly created mods that are not in this list will never spawn.

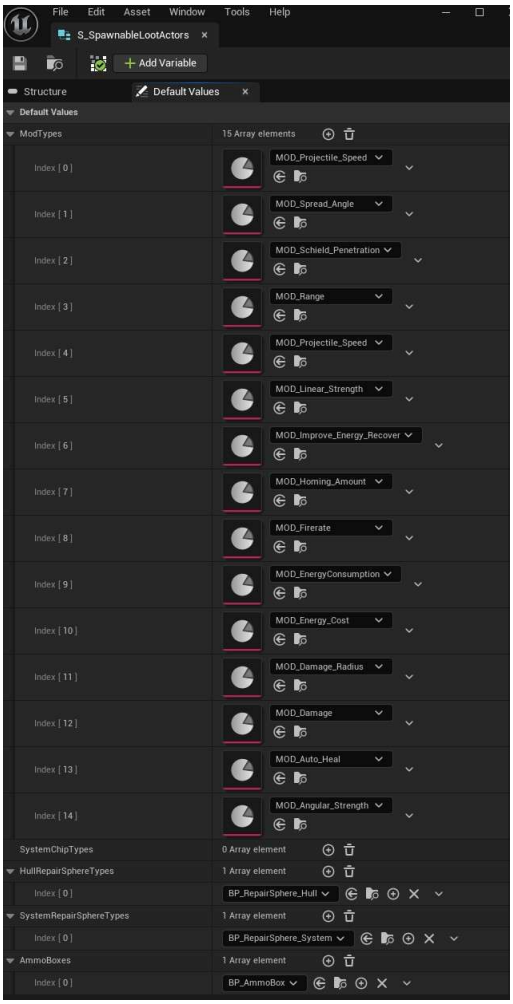


Figure 2 *— Default values of S_SpawnableLootActors*

6 Procedural sector generator

6.1 What it spawns

World elements	Gameplay elements
Dungeons	Loot
Asteroids	Enemies
Debris	Minefields
Planets (empty)	Area enemy spawners (bring in enemies when the player gets close)
Rotating space stations (placeholders for now)	

To do: randomize the planet around the dungeon.

6.2 Generation procedure

Stage	What happens
Game	Prepare the list of unique objects for the whole game
Galaxy	Prepare the list of unique objects for this galaxy
Sector	Prepare the list of unique objects for this sector Pick a random number of hero items (1–2) Pick a random number of common items (3–8)

6.3 Setting up the generator

> **Testing**
> To test the generator, open the `SystemGeneratorTestMap` level or make a copy of it in your developer folder.

Preparation

1. Use the `SystemGeneratorTestMap` map, or a copy of it.
2. In your own folder, create a blueprint starting from `SYSGEM_Example`. This is the set used for generation.
3. Select `BP_SystemGenerator` and, in the inspector, add your blueprint under **System Generator Classes**. If you add more than one, the generator picks one at random.

Editing your set

Open your set (`SYSGEM_<name>`). In each pool, add one actor blueprint per element.

Section	What goes in it
Actor Pool Once Per Sector	Actors that should appear at most once or twice in a sector
Skyboxes pool	**ActorClass:** the skybox blueprint **LightColor:** extra lights in different colors to tint the skybox **LightIntensityMinMax:** light intensity range
Actor Pool Reusable Actors	Actors that can be reused several times
Common Enemies	Enemies that can appear in the sector

6.4 Adding dungeons to a sector

Dungeons are added through `BP_SystemGeneratorGroupUnit_PlanetWithDungeon`.

> **Known crash**
> Do NOT click "Open Full Blueprint Editor" on this blueprint: the editor crashes.

1. Make a copy of the blueprint in your own folder.
2. In the **Decorators** variable, open **Index [0] › GeneratorParams › Data**.
3. Add an element named `DungeonsPool` and set its type (on the right) to `FActor Pool`.
4. Add to the array the **exterior** blueprints of the dungeons you want to use.

6.5 Group compositions

- `BP_SystemGeneratorGroup` creates a composition of objects, called `BP_SystemGeneratorGroupUnit`.
- Inside the group, add child actors and position them in the scene. In each child actor, set **Child Actor Class** to the group unit you want.
- `BP_SystemGeneratorGroupUnit_PlanetWithDungeon` can also be used as a group unit.

> **Placing a dungeon inside an asteroid field**
> In `BP_AsteroidField3_Runtime` › **Position Generator**:
> • tick **UseHole**, so the field has a hole in the center;
> • make sure the minimum and maximum radius are larger than the radius of the dungeon planet (set by hand for now).

6.6 Object spawn radius

In `/Content/Developers/BPFL_SystemGeneratorShared`, open the function **GetStaticBounds** and edit the local variable *StaticBoundsConfig*: add the blueprint class and the actor's **radius** in centimeters.

6.7 Parameter reference

Parameters use the form Name/InstancedStruct and are optional unless stated otherwise.

Random actors and accepted parameters

Actor	Accepted generator params
Asteroids	`UseHole`, `HoleRadiusMultiplierForRadiusMinMax`, `RadiusMinMax`, `CFunctionTypes`
BP_Planet2	—
RandomServerSpawner	`CFunctionTypes`, `RadiusMinMax`, `HoleRadiusMultiplierForRadiusMinMax`, `UseHole`, `ActorPoolParam`
RandomServerLootSpawner	`DropsDataAssetPool`

Generator parameters

Parameter	Description	Default	Type
`RadiusMinMax`	Minimum and maximum radius	50000 – 200000	Vector 2D
`UseHole`	Whether to leave an empty area in the center	Random	Boolean
`HoleRadiusMultiplierForRadiusMinMax`	Min and max of a number multiplied by the maximum radius to get the radius of the empty sphere	0.1 – 0.5	Vector 2D
`CFunctionTypes`	0 Spherical cloud · 1 Flat disc · 2 Ring · 3 Separate clusters · 4 Filament/Spiral (probably not working)	All	FCFunctionT…
`ScaleMinMax`	Limits for the scale (use 0.5 and 0.6)	1 – 1	Vector 2D
`DropsDataAssetPool`	Data assets that can be used for spawning	null	FDropsDataA…
`PositionGenerator`	—	—	FPositionGe…
`RandomServerLootSimple`	If true, spawns a single one in the center	false	Boolean
`ActorPoolParam`	Array of actor classes	empty	AtorPoolPar…